

Golang 源码剖析

《学习笔记》第五版 下册



前言	4
更新	5
一. 准备	6
二. 引导	7
三. 初始化	10
四. 内存分配	17
1. 概述	17
2. 初始化	21
3. 分配	26
4. 回收	39
5. 释放	42
6. 其他	44
五. 垃圾回收	49
1. 概述	49
2. 初始化	51
3. 启动	51
4. 标记	57
5. 清理	66
6. 监控	69
7. 其他	71
六. 并发调度	80
1. 概述	80
2. 初始化	81
3. 任务	85
4. 线程	95
5. 执行	104
6. 连续栈	119
7. 系统调用	132
8. 监控	137
9. 其他	142
七. 通道	152
1. 创建	152
2. 收发	153

3. 选择	161
八. 延迟	169
1. 定义	169
2. 性能	174
3. 错误	175
九. 析构	178
1. 设置	178
2. 清理	182
3. 执行	183
十. 缓存池	188
1. 初始化	188
2. 操作	190
3. 清理	192

前言

我是个安全感匮乏的人，对新鲜事物总会保持一定的警惕。总想知道为什么会这样，为什么会那样，渴望将一切都看得通透，而不仅仅是记住字里行间的规则条理。

知道 Golang 很早，但观望了相当长时间。究其原因，无非是一门新出的语言，自身和相关资源都不成熟，不值得立即投入精力。只是后来屡屡出现的“NextC”让我终究起了一探究竟的欲望，很想知道这个 goroutine 和 coroutine 究竟有什么区别。正好那段时间我在拆解 greenlet 和 lua 的源码，算是相互借鉴。

从 R60 到现在，历经好几年，一直跟着源码去学习。其间有各种故事，倒不值得在此絮叨，只能说欣喜苦恼掺杂，乐在其中罢了。虽说这是本写 Golang 的书，但我依然庆幸自己的 C、ASM 底子不错，让我多了种学习手段，能比多数人了解得更深入些。

尽管这已是本书第五版，但内容几乎全部重写，各种错漏在所难免，希望您能及时指正。

全书共分三册：上册《语言详解》，中册《标准库》（未定），下册《源码剖析》。

联系方式：

微博：weibo.com/qyuhén

开源：github.com/qyuhén/book

邮件：qyuhén@hotmail.com

社区：qyuhén.bearychat.com

雨痕

二〇一五年，冬



更新

- 2012-01-11 开始学习 Go。
- 2012-01-15 第一版，基于 R60。
- 2012-03-29 升级到 1.0。
- 2012-06-15 升级到 1.0.2。
- 2013-03-26 升级到 1.1。
- 2013-12-12 第二版，基于 1.2。
- 2014-05-22 第三版，基于 1.3。
- 2014-12-20 第四版，基于 1.4。
- 2015-06-15 第五版，基于 1.5 RC。
- 2015-11-01 新版《学习笔记》，升级到 1.5.1。
- 2015-12-09 下册《源码剖析》截稿。
- 2015-12-18 下册《源码剖析》校对结束，正式发布。

一. 准备

内容基于 Golang 1.5.1，测试环境 Linux AMD64，不包含 32 位内容。

我觉得是时候抛弃 32 位平台了。除了学习，日常开发和架构都不需要这个东西了。而且运行时内部对 32 位的处理看着就别扭。

本书重点剖析 Golang 运行时的内部执行机制，以便能深入了解程序运行期状态，这有助于深入理解语言规则，写出更好的代码，无论是规避 GC 潜在问题，还是为了节约内存，亦或提升运行性能。

为便于阅读，相关代码被删减，如有疑问请对照原始文件。如果 Golang 版本不同，示例代码行号可能会存在差异，请以您实际测试输出为准。

本书相关环境：

```
$ go version
go version go1.5.1 linux/amd64

$ lsb_release -d
Description:    Ubuntu 14.04.3 LTS

$ gdb --version
GNU gdb (Ubuntu 7.7.1-0ubuntu5~14.04.2) 7.7.1
```

本书示例 go 安装包存放在 /usr/local/go 目录，可能与您的有所不同，不影响测试。

二. 引导

事实上，编译好的可执行文件真正执行入口并非我们所写的 `main.main` 函数，因为编译器总是会插入一段引导代码，完成诸如命令行参数、运行时初始化等工作，然后才会进入用户逻辑。

要从 `src/runtime` 目录下的一堆文件中找到真正的入口，其实很容易。随便准备一个编译好的目标文件，比如 “Hello, World!”。

test.go

```
package main

func main() {
    println("hello, world!")
}
```

编译，然后用 GDB 查看。

建议：尽可能使用命令行编译，而不是某些 IDE 的菜单命令，这有助于我们熟悉各种编译开关参数的具体功能。其次，调试程序时，建议使用 `-gcflags "-N -l"` 参数关闭编译器代码优化和函数内联，避免断点和单步执行无法准确对应源码行，避免小函数和局部变量被优化掉。

```
$ go build -gcflags "-N -l" -o test test.go
```

如果在平台使用交叉编译（Cross Compile），需要设置 `GOOS` 环境变量。

```
$ gdb test

(gdb) info files
Local exec file:
    Entry point: 0x44dd00

(gdb) b *0x44dd00
Breakpoint 1 at 0x44dd00: file /usr/local/go/src/runtime/rt0_linux_amd64.s, line 8.
```

很简单，找到真正的入口地址，然后利用断点命令就可以轻松找到目标源文件信息。

在 `src/runtime` 目录下有很多不同平台的入口文件，都由汇编实现。

```
$ ls rt0_*
rt0_android_arm.s      rt0_dragonfly_amd64.s  rt0_linux_amd64.s      ...
rt0_darwin_386.s       rt0_freebsd_386.s      rt0_linux_arm.s         ...
rt0_darwin_amd64.s     rt0_freebsd_amd64.s    rt0_linux_arm64.s      ...
```

用你习惯的代码编辑器打开源文件，跳转到指定行，查看具体内容。

`rt0_linux_amd64.s`

```
TEXT _rt0_amd64_linux(SB),NOSPLIT,$-8
    LEAQ    8(SP), SI // argv
    MOVQ    0(SP), DI // argc
    MOVQ    $main(SB), AX
    JMP     AX

TEXT main(SB),NOSPLIT,$-8
    MOVQ    $runtime·rt0_go(SB), AX
    JMP     AX
```

用 GDB 设置断点命令看看这个 `rt0_go` 在哪。

注意：源码文件中的 “`·`” 符号编译后变成正常的 “`.`”。

```
(gdb) b runtime.rt0_go
Breakpoint 2 at 0x44a780: file /usr/local/go/src/runtime/asm_amd64.s, line 12.
```

这段汇编代码就是要找的真正目标，正是它完成了初始化和运行时启动。

`asm_amd64.s`

```
TEXT runtime·rt0_go(SB),NOSPLIT,$0

...

// 调用初始化函数。
CALL    runtime·args(SB)
CALL    runtime·osinit(SB)
CALL    runtime·schedinit(SB)

// 创建 main goroutine 用于执行 runtime.main。
MOVQ    $runtime·mainPC(SB), AX
```

```
PUSHQ    AX
PUSHQ    $0
CALL     runtime·newproc(SB)
POPQ     AX
POPQ     AX

// 让当前线程开始执行 main goroutine。
CALL     runtime·mstart(SB)

RET

DATA     runtime·mainPC+0(SB)/8,$runtime·main(SB)
GLOBL    runtime·mainPC(SB),RODATA,$8
```

至此，由汇编针对特定平台实现的引导过程就全部完成。后续内容基本上都是由 Golang 代码实现。

```
(gdb) b runtime.main
Breakpoint 3 at 0x423250: file /usr/local/go/src/runtime/proc.go, line 28.
```

三. 初始化

整个初始化过程相当繁琐，要完成诸如命令行参数整理，环境变量设置，以及内存分配器、垃圾回收器和并发调度器的工作现场准备。

依照前一章找出的线索，先依次看看几个初始化函数的内容。依旧用设置断点命令确定函数所在源文件名和代码行号。

```
(gdb) b runtime.args
Breakpoint 7 at 0x42ebf0: file /usr/local/go/src/runtime/runtime1.go, line 48.

(gdb) b runtime.osinit
Breakpoint 8 at 0x41e9d0: file /usr/local/go/src/runtime/os1_linux.go, line 172.

(gdb) b runtime.schedinit
Breakpoint 9 at 0x424590: file /usr/local/go/src/runtime/proc1.go, line 40.
```

函数 args 整理命令行参数，这个没什么需要深究的。

runtime1.go

```
func args(c int32, v **byte) {
    argc = c
    argv = v
    sysargs(c, v)
}
```

函数 osinit 确定 CPU Core 数量。

os1_linux.go

```
func osinit() {
    ncpu = getproccount()
}
```

最关键的就是 schedinit 这里，几乎我们要关注的所有运行时环境初始化构造都在这里被调用。函数头部的注释列举了启动过程，也就是前一章的内容，不过信息太过简洁了点。

proc1.go

```
// The bootstrap sequence is:
//
```

```
// call osinit
// call schedinit
// make & queue new G
// call runtime·mstart
func schedinit() {
    // 最大系统线程数量限制, 参考标准库 runtime/debug.SetMaxThreads。
    sched.maxmcount = 10000

    // 栈、内存分配器、调度器相关初始化。
    stackinit()
    mallocinit()
    mcommoninit(_g_.m)

    // 处理命令行参数和环境变量。
    goargs()
    goenvs()

    // 处理 GODEBUG、GOTRACEBACK 调试相关的环境变量设置。
    parsedebugvars()

    // 垃圾回收器初始化。
    gcinit()

    // 通过 CPU Core 和 GOMAXPROCS 环境变量确定 P 数量。
    procs := int(ncpu)
    if n := atoi(gogetenv("GOMAXPROCS")); n > 0 {
        if n > _MaxGomaxprocs {
            n = _MaxGomaxprocs
        }
        procs = n
    }

    // 调整 P 数量。
    if procsizes(int32(procs)) != nil {
        throw("unknown runnable goroutine during bootstrap")
    }
}
```

内存分配器、垃圾回收器、并发调度器的初始化细节需要涉及很多专属特征，先不去理会，留待后续章节再做详解。

事实上，初始化操作到此并未结束，因为接下来要执行的是 `runtime.main`，而不是用户逻辑入口函数 `main.main`。

```
(gdb) b runtime.main
Breakpoint 10 at 0x423250: file /usr/local/go/src/runtime/proc.go, line 28.
```

在这里我们关注的焦点是：包初始化函数 `init` 的执行。

proc.go

```
// The main goroutine.
func main() {
    // 执行栈最大限制: 1 GB on 64-bit, 250 MB on 32-bit.
    if ptrSize == 8 {
        maxstacksize = 1000000000
    } else {
        maxstacksize = 250000000
    }

    ...

    // 启动系统后台监控（定期垃圾回收，以及并发任务调度相关）。
    systemstack(func() {
        newm(sysmon, nil)
    })

    ...

    // 执行 runtime 包内所有初始化函数 init。
    runtime_init()

    ...

    // 启动垃圾回收器后台操作。
    gcenable()

    // 执行所有的用户包（包括标准库）初始化函数 init。
    main_init()

    ...

    // 执行用户逻辑入口 main.main 函数。
    main_main()

    // 执行结束，返回退出状态码。
    exit(0)
}
```

与之相关的就是 `runtime_init` 和 `main_init` 这两个函数，它们都是由编译器动态生成。

proc.go

```
//go:linkname runtime_init runtime.init
func runtime_init()

//go:linkname main_init main.init
func main_init()
```

```
//go:linkname main_main main.main
func main_main()
```

注意链接后符号名的变化: runtime_init > runtime.init。

我们准备一个稍微复杂点的示例，看看编译器究竟干了什么。

```
+ <src>
|
+- main.go, test.go
|
+- <lib>
|
+- sum.go
```

lib/sum.go

```
package lib

func init() {
    println("sum.init")
}

func Sum(x ...int) int {
    n := 0
    for _, i := range x {
        n += i
    }

    return n
}
```

test.go

```
package main

import (
    "lib"
)

func init() {
    println("test.init")
}

func test() {
    println(lib.Sum(1, 2, 3))
}
```

main.go

```

package main

import (
    _ "net/http"    // 引入一个标准库里的包。
)

func init() {
    println("main.init.2")
}

func main() {
    test()
}

func init() {
    println("main.init.1")
}

```

编译，执行输出。

```

$ go build -gcflags "-N -l" -o test

$ ./test
sum.init
main.init.2
main.init.1
test.init
6

```

接下来我们用反汇编工具，看看最终动态生成代码的真实面目。

```

$ go tool objdump -s "runtime\.\init\b" test

TEXT runtime.init.1(SB) /usr/local/go/src/runtime/alg.go
    alg.go:322      ...

TEXT runtime.init.2(SB) /usr/local/go/src/runtime/mstats.go
    mstats.go:148  ...

TEXT runtime.init.3(SB) /usr/local/go/src/runtime/panic.go
    panic.go:154   ...

TEXT runtime.init.4(SB) /usr/local/go/src/runtime/proc.go
    proc.go:140    ...

```

```

TEXT runtime.init(SB) /usr/local/go/src/runtime/zversion.go
    zversion.go:9    ...
    panic.go:9      ...
    select.go:45    ...

    zversion.go:9    CALL runtime.init.1(SB)
    zversion.go:9    CALL runtime.init.2(SB)
    zversion.go:9    CALL runtime.init.3(SB)
    zversion.go:9    CALL runtime.init.4(SB)
    zversion.go:9    MOVL $0x2, 0x43f436(IP)
    zversion.go:9    ADDQ $0x58, SP
    zversion.go:9    RET

```

命令行工具 `go tool objdump` 可用来查看实际生成的汇编代码，参数使用正则表达式。当然如果习惯 Intel 格式，那么还是用 GDB 吧。

很显然，`runtime` 内相关的多个 `init` 函数被赋予唯一符号名，然后再由 `runtime.init` 进行统一调用。注意，`zversion.go` 也是动态生成的。

`zversion.go`

```

// auto generated by go tool dist

package runtime

const defaultGoroot = `/usr/local/go`
const theVersion = `go1.5.1`
const goexperiment = ``
const stackGuardMultiplier = 1
var buildVersion = theVersion

```

至于 `main.init`，情况基本一致。区别在于它负责调用非 `runtime` 包的初始化函数。

```

$ go tool objdump -s "main\.\init\b" test

TEXT main.init.1(SB) src/main.go
    main.go:7    ...

TEXT main.init.2(SB) src/main.go
    main.go:15   ...

TEXT main.init.3(SB) src/test.go
    test.go:7    ...

TEXT main.init(SB) src/test.go
    test.go:13   ...
    test.go:13   CALL net/http.init(SB)

```

```
test.go:13 CALL test/lib.init(SB)
test.go:13 CALL main.init.1(SB)
test.go:13 CALL main.init.2(SB)
test.go:13 CALL main.init.3(SB)
test.go:13 MOVL $0x2, 0x48d543(IP)
test.go:13 RET
```

被引用的包，包括 lib 和标准库 net/http 里的 init 函数都被 main.init 调用。

虽然从当前版本的编译器角度来说，init 的执行顺序和依赖关系、文件名以及定义顺序有关。但这种次序非常不便于维护和理解，极易造成潜在错误，所以强烈要求让 init 只做该做的事情：局部初始化。

最后需要记住：

- 所有 init 函数都在同一个 goroutine 内执行。
- 所有 init 函数结束后才会执行 main.main 函数。

四. 内存分配

内置运行时的编程语言通常会抛弃传统的内存分配方式，改由自主管理。这样可以完成类似预分配、内存池等操作，以避开系统调用带来的性能问题。当然，还有一个重要原因是为了更好地配合垃圾回收。

1. 概述

在深入内存分配算法细节前，我们需要了解一些基本概念，这有助于建立宏观认识。

基本策略：

1. 每次从操作系统申请一大块内存（比如 1MB），以减少系统调用。
2. 将申请到的大块内存按照特定大小预先切分成小块，构成链表。
3. 为对象分配内存时，只需从大小合适的链表提取一个小块即可。
4. 回收对象内存时，将该小块内存重新归还到原链表，以便复用。
5. 如闲置内存过多，则尝试归还部分内存给操作系统，降低整体开销。

内存分配器只管理内存块，并不关心对象状态。且不会主动回收内存，由垃圾回收器在完成清理操作后，触发内存分配器回收操作。

内存块

分配器将其管理的内存块分为两种：

- **span**: 由多个地址连续的页（page）组成的大块内存。
- **object**: 将 span 按特定大小切分成多个小块，每个小块可存储一个对象。

按照其用途，span 面向内部管理，object 面向对象分配。

分配器按页数来区分不同大小的 span。比如，以页数为单位将 span 存放到管理数组中，需要时就以页数为索引进行查找。当然，span 大小并非固定不变。在获取闲置 span 时，如果没找到大小合适的，那就返回页数更多的，此时会引发裁剪操作，多余部分将构成新

的 span 被放回管理数组。分配器还会尝试将地址相邻的空闲 span 合并，以构建更大的内存块，减少碎片，提供更灵活的分配策略。

malloc.go

```
_PageShift = 13
_PageSize  = 1 << _PageShift  // 8KB
```

mheap.go

```
type mspan struct {
    next    *mspan    // 双向链表。
    prev    *mspan
    start   pageID   // 起始序号 = (address >> _PageShift)
    npages  uintptr   // 页数
    freelist gclinkptr // 待分配的 object 链表。
}
```

用于存储对象的 object，按 8 字节倍数分为 n 种。比如说，大小为 24 的 object 可用来存储范围在 17 ~ 24 字节的对象。这种方式虽然会造成一些内存浪费，但分配器只需面对有限的几种规格（size class）小块内存，优化了分配和复用管理策略。

分配器会尝试将多个微小对象组合到一个 object 块内，以节约内存。

malloc.go

```
_NumSizeClasses = 67
```

分配器初始化时，会构建对照表存储大小和规格的对应关系，包括用来切分的 span 页数。

msize.go

```
// Size classes. Computed and initialized by InitSizes.
//
// SizeToClass(0 <= n <= MaxSmallSize) returns the size class,
// 1 <= sizeclass < NumSizeClasses, for n.
// Size class 0 is reserved to mean "not small".
//
// class_to_size[i] = largest size in class i
// class_to_allochnpages[i] = number of pages to allocate when
// making new objects in class i

var class_to_size [_NumSizeClasses]int32
```

```
var class_to_allocnpages [_NumSizeClasses]int32

var size_to_class8 [1024/8 + 1]int8
var size_to_class128 [(_MaxSmallSize-1024)/128 + 1]int8
```

若对象大小超出特定阈值限制，会被当做大对象（large object）特别对待。

malloc.go

```
_MaxSmallSize = 32 << 10    // 32KB
```

管理组件

优秀的内存分配器必须要在性能和内存利用率之间做到平衡。好在，Golang 的起点很高，直接采用了 tcmalloc 的成熟架构。

malloc.go

```
// Memory allocator, based on tcmalloc.
// http://goog-perftools.sourceforge.net/doc/tcmalloc.html
```

分配器由三种组件组成。

- **cache**: 每个运行期工作线程都会绑定一个 cache，用于无锁 object 分配。
- **central**: 为所有 cache 提供切分好的后备 span 资源。
- **heap**: 管理闲置 span，需要时向操作系统申请新内存。

mheap.go

```
type mheap struct {
    free      [_MaxMHeapList]mspan // 页数在 127 以内的闲置 span 链表数组。
    freelarge mspan                // 页数大于 127 (>= 1MB) 大 span 链表。

    // 每个 central 对应一种 sizeclass。
    central [_NumSizeClasses]struct {
        mcentral mcentral
    }
}
```

mcentral.go

```

type mcentral struct {
    sizeclass int32    // 规格。
    nonempty  mspan    // 链表: 尚有空闲 object 的 span。
    empty     mspan    // 链表: 没有空闲 object, 或已被 cache 取走的 span。
}

```

mcache.go

```

type mcache struct {
    alloc [_NumSizeClasses]*mspan    // 以 sizeclass 为索引管理多个用于分配的 span。
}

```

分配流程:

1. 计算待分配对象对应规格 (size class)。
2. 从 cache.alloc 数组找到规格相同的 span。
3. 从 span.freelist 链表提取可用 object。
4. 如 span.freelist 为空, 从 central 获取新 span。
5. 如 central.nonempty 为空, 从 heap.free/freelarge 获取, 并切分成 object 链表。
6. 如 heap 没有大小合适的闲置 span, 向操作系统申请新内存块。

释放流程:

1. 将标记为可回收 object 交还给所属 span.freelist。
2. 该 span 被放回 central, 可供任意 cache 重新获取使用。
3. 如 span 已收回全部 object, 则将其交还给 heap, 以便重新切分复用。
4. 定期扫描 heap 里长时间闲置的 span, 释放其占用内存。

注: 以上不包括大对象, 它直接从 heap 分配和回收。

作为工作线程私有且不被共享的 cache 是实现高性能无锁分配的核心, 而 central 的作用是在多个 cache 间提高 object 利用率, 避免内存浪费。

假如 cache1 获取一个 span 后, 仅使用了一部分 object, 那么剩余空间就可能会被浪费。而回收操作将该 span 交还给 central 后, 该 span 完全可以被 cache2、cacheN 获取使用。此时, cache1 已不再持有该 span, 完全不会造成问题。
